PAYFLOW AI

ANTIGRAVITY MASTER PROMPT PLAYBOOK

From an empty folder to a working, judge-ready PayPal + AI hackathon submission

TEAM	PRODUCT	CORE STACK
XyphX 2 members	AI Accounts Receivable Agent Invoice-to-payment workflow	React / Vite + FastAPI MongoDB + PayPal Sandbox

A phased sequence of copy-ready prompts. Run one prompt at a time in Antigravity, inspect the changes, test the acceptance checks, and only then continue.

Version 1.0 • 10 November 2026 • Prepared for the PayPal AI Hackathon

How to use this playbook

This is not one giant prompt. It is a controlled build sequence. Each prompt gives Antigravity a bounded task, expected outputs, and completion checks.

- **Start from an empty project folder.** Open that folder in Antigravity and allow it to inspect the workspace.
- **Paste only one prompt at a time.** Wait for Antigravity to finish, review the files, run the app, and verify the acceptance checks.
- **Do not skip the PayPal sandbox milestone.** The hackathon requires a meaningful, working PayPal integration and meaningful AI use.
- **Do not claim sponsor integrations before using them.** AG Grid, Render, Postman, APIMatic, Bryntum, and Zapier should be selected on Devpost only if they are genuinely integrated or used as required by the rules.
- **Keep secrets out of source control.** Use .env locally, .env.example for placeholders, and deployment secret settings in Render.
- **Use sandbox data only.** Never process real money or store card details.
- **Keep the MVP narrow.** Complete one invoice-to-payment workflow end-to-end before optional features.

Recommended repository layout

```
payflow-ai/
frontend/ # React + Vite + TypeScript
backend/
app/
api/ # FastAPI routers
core/ # config, security, logging
db/ # MongoDB connection
models/ # Pydantic/domain models
services/ # PayPal, AI, invoices
agents/ # tool definitions/orchestration
jobs/ # reminder scheduler
webhooks/ # PayPal webhook handlers
tests/
docs/
.gitignore
LICENSE
README.md
.env.example
```

Product scope and non-negotiable rules

- Core workflow: natural-language instruction → structured invoice draft → human approval → PayPal Sandbox invoice creation/send → webhook/status synchronization → receivables dashboard.
- AI may propose and prepare actions. Backend policy checks and explicit user approval authorize sensitive operations.
- The LLM must never receive PayPal client secrets, database credentials, or unrestricted database access.
- All money amounts use decimal-safe handling (Decimal in Python); never use binary floating-point for financial calculations.
- Store currency explicitly. Do not assume all customers use USD.
- Webhook processing must verify PayPal signatures using the official verification mechanism and handle duplicate deliveries idempotently.
- Do not fabricate payment success. Only verified PayPal API responses or verified webhook events can update payment state.

Product definition and design system

Product statement

PAYFLOW AI is an AI-powered accounts receivable assistant for freelancers and small businesses. It helps users prepare invoices, send them through PayPal, monitor receivables, and prepare contextual follow-ups while keeping the business owner in control.

Visual design system — editorial quiet luxury

Token	Hex	Use
Background	#FAF7F2	Main page canvas
Sand	#D9CFBA	Primary surface / bands
Cream	#EDE6D6	Secondary surface / tables
Ink	#0B0B0B	Primary buttons / feature cards
Muted	#4A4036	Body text / subtle detail
Hairline	#3A342C	1px borders and dividers
Paid	#7A7F5A	Muted olive status
Pending	#B5835A	Muted clay status
Overdue	#8C4A3F	Muted brick status

- Headings: uppercase elegant serif (Cormorant Garamond or Playfair Display). If font loading is impractical, use a close system serif fallback.
- Body/UI: Inter or a clean sans-serif; compact labels with generous letter spacing.
- Square geometry: maximum 2px radius. No pills, glassmorphism, bright gradients, or heavy shadows.
- Cards: black surface, thin inset border, centered uppercase heading where appropriate.
- Buttons: black, uppercase, small text, wide tracking, no shadow.
- Inputs: transparent or cream/sand surfaces with 1px hairline borders.
- Tables: flat cream background, fine row dividers, no zebra striping.
- Decorative wave line art may appear in quiet section corners; never compromise readability.
- Maintain accessible contrast, visible keyboard focus, labels, and responsive layouts.

PROMPT 01 — Workspace inspection and build contract

Objective: Implement this phase exactly as described in the prompt.

```
You are the lead engineer building PAYFLOW AI from scratch in this workspace. Before writing code, inspect the existing folder and report what is present. Do not delete user files. If the folder is empty, create the project. Read the complete project brief below and treat it as the source of truth. Product: PAYFLOW AI, an AI-powered accounts receivable agent for freelancers and small businesses. The core workflow is natural-language invoice request → structured draft → human approval → PayPal Sandbox invoice creation and sending → verified payment-status synchronization → dashboard. Solo/two-person hackathon project. Optimize for a working, reliable MVP in a short time. Do not introduce microservices, Kubernetes, Redis, or complex infrastructure. Preferred stack: React + Vite + TypeScript frontend; FastAPI + Python backend; MongoDB Atlas; PayPal REST APIs in Sandbox; one LLM provider with tool calling. Use the simplest reliable implementation. Design system: editorial quiet luxury. Background #FAF7F2; sand #D9CFBA; cream #EDE6D6; ink #0B0B0B; muted text #4A4036; hairline #3A342C; paid #7A7F5A; pending #B5835A; overdue #8C4A3F. Elegant uppercase serif headings, clean sans-serif UI, square corners (max 2px), no pills, no glassmorphism, no bright gradients, no heavy shadows. Buttons are black uppercase with tracking. Tables are cream with fine dividers. Use whitespace and restrained line-art. Engineering rules: - TypeScript strict mode; Python type hints; Pydantic request/response schemas. - Keep modules small and readable; avoid giant components and giant router files. - Never hardcode secrets. Create .env.example with placeholders. - Never expose PayPal credentials or LLM keys to the browser. - Use Decimal for money, store currency explicitly. - Enforce authenticated user ownership on every business record. - AI may prepare actions but may not bypass server-side validation or approval. - Verify PayPal webhook signatures; process events idempotently. - Do not fabricate API results or payment success. - Add useful errors, loading states, empty states, and tests. - Do not claim sponsor tools are integrated unless they are actually used. First output: concise implementation plan, assumptions, proposed directory tree, dependencies, and milestone order. Then create only the initial scaffold and configuration files. Do not implement all features in one pass.
```

Do not proceed until these checks pass:

- Antigravity reports the actual workspace state before modifying it.
- A written milestone plan and repository tree are produced.
- No secrets or fabricated integrations are added.

PROMPT 02 — Create repository scaffold

Objective: Implement this phase exactly as described in the prompt.

```
Implement the initial repository scaffold for PAYFLOW AI. Use a root folder with frontend/ and backend/ plus docs/. Frontend: Vite + React + TypeScript with strict mode. Backend: FastAPI using a clean app package. Add .gitignore, README skeleton, LICENSE placeholder only if the user chooses the license (recommend MIT for this hackathon), and .env.example files. Do not put real credentials in any file. Backend layout: backend/app/main.py backend/app/core/config.py backend/app/core/logging.py backend/app/api/ backend/app/db/ backend/app/models/ backend/app/schemas/ backend/app/services/ backend/app/agents/ backend/app/jobs/ backend/app/webhooks/ backend/tests/ Frontend layout: frontend/src/app/ frontend/src/components/ frontend/src/features/dashboard/ frontend/src/features/assistant/ frontend/src/features/invoices/ frontend/src/features/customers/ frontend/src/features/approvals/ frontend/src/features/analytics/ frontend/src/lib/ frontend/src/styles/ Add scripts for local development. Pin compatible dependency ranges. Add a /health endpoint that returns service status without exposing secrets. Add a frontend app shell with a clear loading/error boundary. Document Windows PowerShell commands for setup and running both services. Keep the scaffold minimal and runnable.
```

Do not proceed until these checks pass:

- Frontend starts with npm run dev.
- Backend starts with uvicorn app.main:app --reload from backend.
- GET /health returns a simple healthy response.
- No secrets are committed.

PROMPT 03 — Environment configuration and safe settings

Objective: Implement this phase exactly as described in the prompt.

```
Implement typed configuration for FastAPI using pydantic-settings. Read environment variables for APP_ENV, APP_SECRET_KEY, MONGODB_URI, MONGODB_DATABASE, FRONTEND_ORIGIN, PAYPAL_BASE_URL, PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET, PAYPAL_WEBHOOK_ID, AI_PROVIDER, AI_API_KEY, AI_MODEL, and optional EMAIL_PROVIDER settings. Default PayPal base URL to the official Sandbox API host, not Live. Fail safely with clear startup messages when required configuration is absent, but allow /health and tests to run in a documented development mode where external integrations are disabled. Create backend/.env.example with fake placeholders only. Ensure config values are never returned by endpoints or printed in logs. Configure CORS for the local frontend origin from settings, not wildcard in production. Add a settings test that proves secrets are not serialized in public health responses. Document how to set environment variables in Windows PowerShell and Render dashboard.
```

Do not proceed until these checks pass:

- Missing settings produce actionable messages.
- Secrets are masked and never exposed through API responses.
- CORS is restricted to configured origins.

PROMPT 04 — MongoDB connection and persistence layer

Objective: Implement this phase exactly as described in the prompt.

```
Implement an asynchronous MongoDB connection using a maintained Python MongoDB driver supported by the chosen FastAPI stack. Prefer a simple, stable approach and document the choice. Add startup/shutdown lifecycle management, ping verification, indexes, and clean dependency injection for database access. Do not create a repository abstraction framework that adds needless complexity. Collections required for MVP: users, customers, invoices, approvals, reminders, transactions, audit_logs, webhook_events. Create indexes for user_id, invoice IDs, PayPal IDs, invoice status, due_date, and webhook event ID where appropriate. Add a small database health check that reports connected/disconnected without returning URI or credentials. Add tests using a mock or isolated test database; never run destructive tests against production data. Document local MongoDB Atlas setup and IP/network access considerations.
```

Do not proceed until these checks pass:

- App starts and shuts down database connection cleanly.
- Indexes are created idempotently.
- Health output contains no connection string.
- Tests do not require production credentials.

PROMPT 05 — Domain models and money rules

Objective: Implement this phase exactly as described in the prompt.

```
Define Pydantic schemas and persistence models for User, Customer, Invoice, Approval, Reminder, Transaction, AuditLog, and WebhookEvent. Use explicit fields, validation, timestamps, and ownership fields. Money must use Decimal with precision and currency code (ISO 4217). Reject negative invoice totals and unsupported/invalid currency codes. Do not calculate money using float. Define invoice statuses such as draft, pending_approval, created, sent, unpaid, partially_paid (only if supported by verified data), paid, overdue, cancelled, and failed. Define clear allowed transitions; do not allow arbitrary status updates from the frontend. Invoice fields should include internal_id, user_id, customer_id, invoice_number, description/items, subtotal, tax, total, currency, issue_date, due_date, status, paypal_invoice_id, paypal_invoice_url if returned, sent_at, paid_at, created_at, updated_at. Store only necessary customer details. Add serialization tests for Decimal and dates.
```

Do not proceed until these checks pass:

- Money round-trips without float conversion.
- Every record is scoped to a user.
- Invalid transitions and negative amounts are rejected.
- Schemas have examples and clear validation errors.

PROMPT 06 — Authentication and user ownership

Objective: Implement this phase exactly as described in the prompt.

```
Implement simple secure authentication suitable for a hackathon MVP. Use password hashing with a modern password-hashing library (Argon2id preferred) and short-lived access JWTs. If refresh tokens are implemented, rotate and store hashed refresh tokens; otherwise keep the MVP to access tokens and require re-login. Add register, login, current-user endpoints. Validate email, enforce reasonable password rules, and return generic authentication errors. Create a get_current_user dependency. Every customer, invoice, approval, reminder, transaction, and audit query must include the authenticated user_id. Never trust a user_id supplied by the browser. Add tests proving User A cannot read or mutate User B's records. Add a demo-user seed script only for local/demo use, disabled by default. Document that demo credentials are separate from PayPal Sandbox buyer credentials.
```

Do not proceed until these checks pass:

- Registration and login work.
- Passwords are hashed, never stored in plaintext.
- Cross-user access tests fail safely with 404/403.
- JWT signing secret is environment-only.

PROMPT 07 — Customer management APIs

Objective: Implement this phase exactly as described in the prompt.

```
Implement customer CRUD endpoints under /api/customers. Fields: name, email, optional company, billing address fields only if needed by PayPal invoice creation, notes, created_at, updated_at. Validate email and prevent duplicate customer records for the same user where practical. Add pagination, search by name/email, and stable sorting. Do not expose PayPal credentials or store card/bank information. Create service, router, schemas, and tests. Ensure every query is filtered by current user. Add frontend-ready response shapes and OpenAPI descriptions. Include deletion behavior: prevent deleting a customer with active invoices, or use an archive flag rather than hard deletion.
```

Do not proceed until these checks pass:

- CRUD endpoints pass tests.
- Pagination and search work.
- Ownership isolation is enforced.
- Customer with linked invoices cannot be destructively removed.

PROMPT 08 — Invoice domain service and APIs

Objective: Implement this phase exactly as described in the prompt.

```
Implement invoice draft creation and management before connecting PayPal. Endpoints: POST /api/invoices, GET /api/invoices, GET /api/invoices/{id}, PATCH /api/invoices/{id} for editable drafts only, POST /api/invoices/{id}/submit-for-approval. Include filters for status, customer, due date, overdue, and search. Generate human-readable invoice numbers that are unique per user without relying on timestamps alone. Validate customer ownership, currency, line items, quantity, unit price, tax, due date, and total. Calculate totals server-side using Decimal; ignore any client-supplied total if it does not match. Keep an immutable snapshot of the approved invoice payload before sending to PayPal. Add audit log events for creation and edits. Write tests for rounding, invalid data, duplicate submissions, and ownership.
```

Do not proceed until these checks pass:

- Draft invoice can be created and retrieved.
- Server-side totals are authoritative.
- Only drafts can be edited.
- Invoice number uniqueness and audit logging are tested.

PROMPT 09 — Frontend foundation and editorial design tokens

Objective: Implement this phase exactly as described in the prompt.

```
Build the frontend shell in React + Vite + TypeScript using the exact editorial quiet-luxury theme. Use CSS variables for all colors, typography, borders, spacing, and shadows. Background #FAF7F2; sand #D9CFBA; cream #EDE6D6; ink #0B0B0B; muted #4A4036; hairline #3A342C; paid #7A7F5A; pending #B5835A; overdue #8C4A3F. Use serif uppercase headings and Inter-like sans-serif body/UI. Use local/system fallbacks if remote fonts fail. No pills, no glassmorphism, no bright gradients, no rounded cards (max 2px radius), no heavy shadows. Buttons are black uppercase with letter spacing. Inputs use hairline borders. Tables use cream backgrounds and fine dividers. Create responsive layout with left navigation/sidebar on desktop and accessible mobile navigation. Add subtle decorative wave line-art only as low-contrast SVG/CSS, not as distracting background. Build reusable components: Button, TextInput, Select, TextArea, DataTable, StatusLabel (square/flat, not pill), PageHeader, EmptyState, LoadingState, ErrorState, ConfirmDialog, Toast/Alert. Respect keyboard navigation, focus rings, semantic labels, reduced motion, and contrast.
```

Do not proceed until these checks pass:

- All pages use design tokens rather than scattered hex values.
- Layout works at mobile, tablet, and desktop widths.
- No forbidden visual styles appear.
- Keyboard focus and labels are visible.

PROMPT 10 — App navigation and authentication screens

Objective: Implement this phase exactly as described in the prompt.

```
Implement frontend routing and authentication screens: Login, Register, Dashboard, AI Assistant, Invoices, Customers, Approvals, Analytics, Settings. Protect authenticated routes. Add an API client that attaches access tokens safely and handles 401 responses. Avoid localStorage for sensitive long-lived secrets; choose a documented token strategy appropriate to the backend and deployment. If using httpOnly cookies, configure secure cookie and CORS/credentials correctly. Add clear loading, empty, and error states. Visual direction: generous whitespace, warm ivory canvas, alternating sand/cream bands, black editorial feature panels, uppercase serif headings, small letter-spaced labels. Do not invent product metrics; dashboard should show zero/empty states until real records exist. Add a small product wordmark area and calm navigation.
```

Do not proceed until these checks pass:

- Register/login UI calls real backend endpoints.
- Protected routes redirect unauthenticated users.
- Empty states do not display fabricated data.
- Responsive navigation works.

PROMPT 11 — Dashboard with real receivables data

Objective: Implement this phase exactly as described in the prompt.

```
Build the dashboard using backend data only. Add summary blocks for total outstanding, overdue amount, collected this month, and invoice count. Clearly define calculations and currency handling; if multiple currencies exist, show per-currency totals rather than incorrectly summing them. Add recent invoices and an attention-needed section. Add a small collections trend only if sufficient historical data exists; otherwise show a helpful empty state. Create backend aggregate endpoints with user scoping and tests. Use muted status colors: olive for paid, clay for pending, brick for overdue. The black feature card should use centered uppercase text with an inset hairline border. Keep the page calm and premium, not a generic admin template.
```

Do not proceed until these checks pass:

- Dashboard totals reconcile with invoice records.
- Currencies are not mixed.
- No demo/fake values appear in real mode.
- Loading, error, and empty states are designed.

PROMPT 12 — Invoice list, detail, and creation UI

Objective: Implement this phase exactly as described in the prompt.

```
Implement the Invoices feature. Build a flat cream data table with columns: invoice number, customer, amount/currency, issue date, due date, status, actions. Use AG Grid only if we actually install and integrate it as a meaningful data grid; if used, configure sorting, filtering, pagination, column sizing, and accessible keyboard interactions. Otherwise use a simple accessible table and do not claim AG Grid sponsorship. Build invoice creation form with customer selector, line items, quantity, unit price, currency, issue date, due date, notes. Calculate preview in UI for convenience but backend remains authoritative. Add invoice detail page with activity timeline and PayPal reference when available. Keep draft, pending approval, sent, paid, overdue, and failed states distinct.
```

Do not proceed until these checks pass:

- Create/edit draft works.
- Backend validation errors are shown clearly.
- Tables are usable on narrow screens.
- AG Grid is selected for Devpost only if genuinely integrated.

PROMPT 13 — Approval center and policy engine

Objective: Implement this phase exactly as described in the prompt.

```
Implement human approval as a first-class backend capability. An approval record contains user_id, action_type, resource_type, resource_id, immutable payload snapshot, payload_hash, status (pending/approved/rejected/expired), created_at, approved_at, and execution result reference. Approval must be bound to the authenticated user and exact payload hash. If invoice contents change after approval, invalidate the approval and require a new one. Create endpoints to list, inspect, approve, and reject pending actions. Only the owner who created/owns the invoice can approve. Approval should not itself call PayPal until the execution endpoint validates the approval and policy. Prevent replay and double execution with idempotency keys and status transitions. Add an audit record for each decision. Build a premium Approval Center UI with clear action summary, amount/currency, customer, and exact action being approved.
```

Do not proceed until these checks pass:

- Changing approved payload invalidates approval.
- Approvals cannot be reused or replayed.

- Unauthorized users cannot approve.
- Approval and execution are separate steps.

PROMPT 14 — PayPal Sandbox developer setup

Objective: Implement this phase exactly as described in the prompt.

```
Prepare PayPal integration using the official PayPal Developer Platform and Sandbox only. Do not guess endpoint paths or payload fields. First consult current official PayPal REST API documentation and the current Invoicing API reference. Record the exact endpoints, required fields, supported currencies, OAuth token flow, invoice lifecycle, and webhook verification method in docs/paypal-integration.md. If browser access or docs are unavailable, ask the developer to provide the relevant official docs rather than inventing details. Use environment variables PAYPAL_BASE_URL, PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET, PAYPAL_WEBHOOK_ID. The Sandbox base URL should be the official PayPal Sandbox API host. Implement OAuth client-credentials token acquisition server-side, token caching only in memory with expiry safety, timeouts, bounded retries for safe requests, and structured redacted logs. Never log access tokens, client secrets, payer personal data, or full sensitive payloads. Create a PayPal service module and a small connection diagnostic endpoint that reports connected/not-connected without revealing credentials. Add mocked tests and a manual sandbox test guide.
```

Do not proceed until these checks pass:

- Official docs are referenced in implementation notes.
- OAuth credentials remain server-side.
- No invented API fields or endpoints.
- Sandbox is the default and only environment for hackathon demo.

PROMPT 15 — PayPal invoice creation adapter

Objective: Implement this phase exactly as described in the prompt.

```
Implement the PayPal Invoicing API adapter based on the verified official documentation recorded in docs/paypal-integration.md. Map our internal invoice model to the official PayPal invoice payload. Support only fields verified by the current API docs. Handle API validation errors and map them to safe user-facing messages. Persist PayPal invoice ID and any official invoice URL returned by the API. Keep internal status separate from PayPal status and map statuses only according to documented values. Add POST /api/paypal/invoices or an internal service method that can only be called by an approved invoice execution. Use a stable idempotency/request identifier if the API supports it. Never create an invoice twice on a network retry without checking existing execution state. Add mocked unit tests and one manual Sandbox test procedure. Do not send an invoice automatically before user approval.
```

Do not proceed until these checks pass:

- Sandbox invoice is created from a real API response.
- PayPal ID and URL are persisted.
- Duplicate execution is prevented.
- Errors are safe and actionable.

PROMPT 16 — Send invoice through PayPal

Objective: Implement this phase exactly as described in the prompt.

```
Implement the approved invoice sending workflow using the official PayPal Invoicing API operation for sending an existing invoice. Confirm the invoice is in a sendable PayPal status before calling. Require a valid approval tied to the current invoice snapshot. Record send attempt, response status, PayPal invoice ID, timestamp, and audit entry. Handle timeouts carefully: if outcome is uncertain, retrieve invoice status before retrying. Do not report success unless PayPal confirms it. Add a frontend confirmation view showing customer, amount, currency, due date, and PayPal destination/URL if returned. Provide an explicit "Approve and send via PayPal Sandbox" action. Make sandbox labeling obvious so judges know no real money is involved. Write tests for rejected approval, duplicate send, timeout/unknown outcome, and success.
```

Do not proceed until these checks pass:

- Only approved invoices can be sent.

- Successful state comes from PayPal confirmation.
- Unknown outcomes are reconciled before retry.
- UI clearly labels Sandbox.

PROMPT 17 — AI provider abstraction and structured output

Objective: Implement this phase exactly as described in the prompt.

```
Add one AI provider integration using a provider-supported SDK and structured tool/function calling. Read provider/model/API key from environment. Do not hardcode a provider or key. Create a small service interface so the provider can be swapped without rewriting business logic. Set reasonable request timeout, token limits, and safe error handling. Never send PayPal secrets, auth tokens, passwords, or unnecessary customer data to the LLM. The assistant must return validated structured intent for invoice creation and queries. Use Pydantic models for tool arguments. If the model output is incomplete or ambiguous, ask a clarifying question rather than guessing amount, currency, customer, or due date. Do not let model output directly execute writes. Add tests with a fake model client so core tests run without an API key.
```

Do not proceed until these checks pass:

- Provider is configurable via environment.
- Tool arguments are schema-validated.
- Ambiguous requests trigger clarification.
- LLM never gets secrets or direct database access.

PROMPT 18 — AI assistant tool calling

Objective: Implement this phase exactly as described in the prompt.

```
Implement the AI assistant as one orchestrator with a small allowlist of backend tools. Required tools: search_customers, create_invoice_draft, list_invoices, get_overdue_invoices, get_invoice_detail, prepare_reminder_draft, get_receivables_summary, request_approval. Tools must call application services, not raw database access. Each tool must enforce current user scope and validate arguments. The model may choose a read-only tool or prepare a draft. It must not approve or execute a PayPal operation. For sensitive actions, it can create an approval request after showing a preview, but the user must explicitly approve via the UI. Add limits on tool-call count per user turn, protect against prompt injection in customer notes/invoice descriptions, and treat external text as untrusted data. Store conversation history minimally and scope it to the user. Add test scenarios for valid commands, missing customer, ambiguous currency, malicious invoice description, and attempts to bypass approval.
```

Do not proceed until these checks pass:

- Natural language can list invoices and prepare a draft.
- No AI tool can execute PayPal directly.
- All tools enforce user scope.
- Prompt injection does not override system policy.

PROMPT 19 — AI invoice extraction and preview

Objective: Implement this phase exactly as described in the prompt.

```
Implement the natural-language invoice creation flow. Example: 'Create a $500 invoice for John Smith for website development, due in seven days.' The AI should extract customer identity, amount, currency, description, and due date. Resolve customer against the authenticated user's customer list. If multiple matches exist, ask which one. If currency or due date is ambiguous, ask a question. Convert relative dates using the user's configured timezone; if no timezone is configured, ask or use a clearly disclosed default. Do not infer tax or line items unless requested. Return a structured draft preview. Show all extracted fields and computed totals. Let user edit before submitting. Backend validates and creates draft. The frontend must distinguish 'AI suggestion' from 'saved invoice'. Add examples and tests for ambiguous names, missing currency, invalid dates, and natural language variations.
```

Do not proceed until these checks pass:

- AI extraction produces editable preview.

- Ambiguous customer/date/currency asks clarification.
- Backend revalidates all values.
- No invoice is sent during extraction.

PROMPT 20 — Receivables queries and analytics

Objective: Implement this phase exactly as described in the prompt.

```
Implement read-only AI analysis backed by actual database aggregates. Add tools/endpoints for overdue invoices, aging buckets (0–7, 8–30, 31–60, 60+ days), outstanding totals by currency, collections in a selected period, and customer balances. Ensure timezone-aware due-date comparisons. AI summaries must cite the underlying invoice numbers and amounts in the response, and state when there is insufficient data. Do not predict payment dates or probability unless there is enough historical data and the result is explicitly labeled as an estimate. Build an Analytics page with filters and charts/tables. If AG Grid is integrated, use it for meaningful invoice/customer data exploration, not merely a decorative table. Do not combine different currencies into one total. Add test fixtures and reconciliation tests.
```

Do not proceed until these checks pass:

- Every displayed metric reconciles to underlying records.
- Currency grouping is correct.
- AI analysis uses backend aggregates, not invented values.
- Empty datasets show helpful states.

PROMPT 21 — Reminder policy and message generation

Objective: Implement this phase exactly as described in the prompt.

```
Implement payment reminder preparation. Define stages: upcoming (configurable, e.g. 1 day before due date), due today, 1–3 days overdue, 4–7 days overdue, 8–14 days overdue, and 15+ days overdue. These are default business rules, configurable per user. Exclude paid, cancelled, disputed, or already-reminded invoices according to policy. Use idempotency to avoid duplicate reminders for the same invoice and stage. AI generates a concise, respectful, customer-specific reminder using only verified invoice data. Do not threaten, shame, or claim legal action. The message must include invoice number, amount/currency, due date, and a payment link only if a valid PayPal invoice URL exists. Store reminder as draft first. Do not send messages through an unconfigured channel. Add user preview/edit/approve flow and tests for each stage.
```

Do not proceed until these checks pass:

- Reminder stage is deterministic.
- No duplicate reminder for same stage.
- AI message contains only verified data.
- Drafting a reminder does not send it.

PROMPT 22 — Reminder scheduler and delivery channel

Objective: Implement this phase exactly as described in the prompt.

```
Add a simple background scheduler suitable for a single-instance hackathon deployment. Use APScheduler or a simple Render-compatible scheduled job, choosing the most reliable low-complexity option. Document that in-memory schedulers can duplicate or miss jobs across multiple instances; use database idempotency and a single scheduler instance. The scheduler should identify due reminder candidates and create drafts or notifications, not silently send unapproved customer communications. For delivery, choose one real channel only if credentials are available (for example a transactional email provider). If no email provider is configured, keep the reminder in the app's Approval Center and label it 'Ready for approval' rather than pretending it was sent. If Zapier is genuinely used, trigger a Zapier webhook only after the user approves the specific reminder and validate the response. Add retries, dead-letter/error status, and audit logs.
```

Do not proceed until these checks pass:

- Scheduler is safe against duplicate runs.
- No reminder is sent without configured channel and required approval.

- Unconfigured channel is clearly shown.
- Failures are visible and retryable.

PROMPT 23 — PayPal webhook verification and reconciliation

Objective: Implement this phase exactly as described in the prompt.

```
Implement PayPal webhook handling using the official current PayPal webhook verification procedure. Verify the webhook signature using the configured webhook ID and official verification endpoint or supported SDK method. Do not trust a raw request body merely because it contains a PayPal-looking event. Persist each verified event ID before applying effects; unique index prevents duplicate processing. Store minimal event metadata and redacted payload only as needed for audit/debugging. Map only documented invoice/payment event types to internal state transitions. When event order is unexpected or the event lacks sufficient details, retrieve the current invoice status from PayPal before updating. Never mark an invoice paid based solely on an unverified event. Add replay protection, idempotent processing, and tests for invalid signature, duplicate event, unknown event, and valid payment event.
```

Do not proceed until these checks pass:

- Invalid webhook is rejected.
- Duplicate event does not duplicate transaction or audit effect.
- Only verified data changes payment state.
- Unknown events are safely ignored/logged.

PROMPT 24 — Transaction ledger and audit trail

Objective: Implement this phase exactly as described in the prompt.

```
Implement a transaction record and append-only-style audit log. Do not describe the log as legally immutable unless infrastructure truly guarantees immutability. Each event should record actor (user/system/webhook), action, resource, timestamp, result, correlation ID, and safe metadata. Never store secrets or full sensitive payloads. Record invoice created, edited, approved, rejected, PayPal create/send response, webhook received/verified, payment status changed, reminder prepared/approved/sent, and errors. Provide a user-visible activity timeline on invoice details. Make audit records append-only at the application layer: no update/delete endpoints. Add tests that ensure business APIs cannot modify or delete audit records.
```

Do not proceed until these checks pass:

- Activity timeline is traceable.
- Audit log contains no secrets.
- No public mutation endpoint exists for audit records.
- System and user actions are distinguishable.

PROMPT 25 — AG Grid meaningful sponsor integration

Objective: Implement this phase exactly as described in the prompt.

```
If AG Grid is selected as a sponsor integration, integrate the actual AG Grid library into the invoice or customer analytics page. Use the appropriate React package and follow its current licensing requirements. Do not use a fake imitation or claim AG Grid if the app uses a basic HTML table. Build a meaningful grid with sorting, filtering, column visibility, pagination, currency-aware formatting, status rendering, and responsive sizing. Use the editorial quiet-luxury theme: cream grid surface, ink text, hairline dividers, restrained status colors, no bright default blue theme. Keep accessibility and keyboard navigation. Document precisely where AG Grid is used and why it helps users inspect receivables. Add screenshots and a short note in README. If licensing or setup blocks integration, stop and report it; do not falsely claim sponsor usage.
```

Do not proceed until these checks pass:

- Actual AG Grid package is imported and rendered.
- Grid supports meaningful sorting/filtering.
- Theme matches PAYFLOW design.
- README documents the actual integration.

PROMPT 26 — Bryntum calendar integration — optional

Objective: Implement this phase exactly as described in the prompt.

```
Only if time remains after the complete invoice-to-payment workflow is reliable, evaluate Bryntum's current product and licensing for a calendar/scheduler view. If selected, use the actual Bryntum component to visualize invoice due dates and reminder schedule. Do not use it merely as decoration. Show invoice number, customer, due date, status, and click-through to invoice details. Use muted status colors and the quiet-luxury theme. Do not schedule automatic customer messages from calendar drag/drop without a confirmation step. If Bryntum setup, license, or integration takes too long, skip it and do not select it on Devpost. Document actual usage only.
```

Do not proceed until these checks pass:

- Actual Bryntum package/component is present if claimed.
- Calendar data comes from backend records.
- No unapproved communication can be scheduled.

PROMPT 27 — Postman API collection

Objective: Implement this phase exactly as described in the prompt.

```
Create a Postman collection for the real PAYFLOW AI API. Include auth, customers, invoices, assistant, approvals, reminders, analytics, PayPal connection status, and webhook testing guidance. Use collection variables for baseUrl and access token; never include real secrets or real user data. Add example requests and expected responses. For PayPal webhooks, document how to test verified sandbox events using official PayPal tools rather than pretending a manually posted JSON body is verified. Export the collection to docs/postman/PayflowAI.postman_collection.json and add it to the public repo. If Postman is used only for testing, describe it accurately on Devpost.
```

Do not proceed until these checks pass:

- Collection imports successfully.
- No credentials are embedded.
- Core API flows are documented and testable.

PROMPT 28 — APIMatic Context Plugin usage and record

Objective: Implement this phase exactly as described in the prompt.

```
Use the APIMatic PayPal Context Plugin in the actual development workflow if it is available and supported by the coding environment. Follow the official installation instructions from APIMatic rather than relying on an unverified command. Ask the context plugin for PayPal Invoicing API endpoint details, request schemas, response schemas, and webhook references. Compare its output with current official PayPal documentation before coding. Do not blindly trust generated context. Create docs/apimatic-usage.md documenting: installation method used, how the context was queried, what useful API context it provided, any errors or gaps, and how the team validated the final implementation. If it was not successfully used, do not claim it on Devpost.
```

Do not proceed until these checks pass:

- Plugin was actually invoked during development.
- API details were cross-checked against official docs.
- Usage note accurately reports experience.

PROMPT 29 — Zapier integration — optional notification automation

Objective: Implement this phase exactly as described in the prompt.

```
If using Zapier, implement one narrow, real workflow: after an authenticated business owner approves a reminder, the backend sends a signed/secret-protected event to a configured Zapier Catch Hook URL. The Zap may notify the business owner (not the customer unless a proper approved customer delivery channel is configured) through an available app such as email or Slack. Add timeout, retry, signature/shared-secret validation where feasible, and a delivery status. Never expose the Zapier webhook URL to the browser. Never trigger Zapier from raw LLM output. Document the exact Zap trigger/action and include a redacted screenshot or description. If no working Zap is created, do not select Zapier on Devpost.
```

Do not proceed until these checks pass:

- Zapier receives a real approved event.
- Webhook URL is server-only.
- AI cannot trigger arbitrary Zapier actions.
- README explains the actual workflow.

PROMPT 30 — Render deployment preparation

Objective: Implement this phase exactly as described in the prompt.

```
Prepare deployment for Render. Keep one FastAPI web service and one static frontend service or a single-service deployment if simpler and reliable. Use a production start command with Uvicorn/Gunicorn as appropriate for the chosen setup. Add health checks, environment variables, allowed CORS origins, MongoDB Atlas network configuration, and build commands. Do not commit .env. Configure secrets through Render dashboard. Ensure PayPal remains Sandbox. Add a deployment section to README. If background scheduler runs in the web process, document single-instance limitation and prevent multiple replicas. Prefer a separate Render cron job for scheduled candidate generation if available and appropriate. Do not claim Render sponsor integration until deployed on Render.
```

Do not proceed until these checks pass:

- Hosted frontend and backend load.
- Health check passes.
- Secrets are configured outside Git.
- CORS and MongoDB connectivity work in production.

PROMPT 31 — End-to-end integration test

Objective: Implement this phase exactly as described in the prompt.

```
Create a repeatable end-to-end test plan and run it against a test database and PayPal Sandbox. Test: register/login; create customer; issue natural-language invoice command; AI extracts fields; user edits preview; draft is saved; approval is created; user approves; backend creates PayPal invoice; user sends invoice; PayPal response is persisted; webhook signature is verified; duplicate webhook is ignored; dashboard reflects the verified status. Also test rejection, missing currency clarification, duplicate send prevention, invalid webhook, and cross-user access. Use sandbox accounts and clearly document how judges can create their own sandbox buyer account if needed. Do not place private PayPal buyer credentials in public README. Provide a controlled demo account with no real financial data if supported. Record test outcomes and known limitations.
```

Do not proceed until these checks pass:

- Happy path completes end-to-end.
- Negative/security tests pass.
- Sandbox behavior is reproducible.
- Known limitations are written down.

PROMPT 32 — UI polish and responsive quality pass

Objective: Implement this phase exactly as described in the prompt.

```
Perform a complete UI quality review against the quiet-luxury specification. Inspect every route at desktop, tablet, and mobile sizes. Remove generic dashboard styling, excessive cards, pills, rounded corners, gradients, shadows, oversized icons, and bright colors. Make typography hierarchy intentional: large uppercase serif page titles, small tracked labels, readable sans-serif body. Use alternating sand and cream bands, black feature panels with inset hairline borders, transparent/hairline inputs, flat tables, and restrained muted statuses. Check alignment, spacing, empty states, validation messages, loading skeletons, errors, confirmation dialogs, hover/focus states, and mobile table behavior. Add a restrained wave-line SVG in selected section corners only. Do not add decorative elements that harm readability. Take screenshots of all major flows for the submission gallery.
```

Do not proceed until these checks pass:

- All main pages match the requested design system.
- No layout overflow on mobile.
- No fake data or dead controls.
- Every action has loading/success/error feedback.

PROMPT 33 — Security review and abuse cases

Objective: Implement this phase exactly as described in the prompt.

```
Conduct a security review focused on this MVP. Check authentication, authorization, user data isolation, CORS, CSRF implications if cookies are used, password hashing, JWT expiry, rate limiting on login and AI endpoints, prompt injection, LLM data minimization, PayPal secret handling, webhook verification, idempotency, audit records, input validation, HTML escaping, file upload absence, and dependency vulnerabilities. Ensure the frontend cannot set invoice status or approval state directly. Add rate limits to expensive AI calls and prevent unbounded conversation/tool loops. Validate URLs and do not fetch arbitrary URLs supplied by an LLM. Do not log customer PII unnecessarily. Write docs/security.md with threat model and mitigations. Fix critical/high findings before final submission; clearly list any remaining limitations.
```

Do not proceed until these checks pass:

- No critical/high obvious vulnerabilities remain.
- Client cannot bypass backend policy.
- Webhook verification and user isolation tests pass.
- Secrets and PII are not leaked in logs.

PROMPT 34 — README, license, and public repository readiness

Objective: Implement this phase exactly as described in the prompt.

```
Prepare the public GitHub repository for judges. Ensure it includes an OSI-approved license file at repository root (use MIT unless the team chooses another compatible license), detailed README, architecture diagram (Mermaid is acceptable), setup instructions for Windows PowerShell and macOS/Linux, environment variable reference, MongoDB Atlas setup, AI provider setup, PayPal Sandbox setup, run commands, testing instructions, API docs, deployment instructions, screenshots, known limitations, and a clear explanation of exactly how PayPal and AI are used. Document sponsor integrations only when real. Do not publish secrets, .env files, sandbox passwords, customer data, or private keys. Add a clean .env.example. Verify a fresh clone can follow the instructions. Include team name XyphX and team members as appropriate.
```

Do not proceed until these checks pass:

- License is visible at repository root.
- README explains PayPal and AI integration clearly.
- Fresh-clone setup is tested.
- No secrets or private data are committed.

PROMPT 35 — Demo data and judge testing experience

Objective: Implement this phase exactly as described in the prompt.

```
Create a safe demo setup path. Provide a seed script that creates clearly labeled fictional customers and invoices in a local/demo database. Do not create fake PayPal payment confirmations. If judges use the hosted demo, provide a demo account only if it is safe and isolated. Make it obvious which actions call PayPal Sandbox. Provide testing instructions that walk judges through the strongest 2–3 minute scenario. The demo story: a business owner asks for a $500 website-development invoice for a fictional customer; AI extracts details; owner reviews and approves; PayPal Sandbox creates and sends the invoice; an actual sandbox status update or verified webhook updates the invoice; AI prepares a contextual reminder for a separate overdue sample invoice. If webhook/payment completion cannot be demonstrated reliably, show the verified current sandbox invoice state and disclose the limitation. Never fake a successful payment.
```

Do not proceed until these checks pass:

- Demo data is fictional and clearly labeled.

- No fake payment events are inserted.
- Judge flow is short and reproducible.
- Sandbox labels are visible.

PROMPT 36 — Project screenshots and thumbnail assets

Objective: Implement this phase exactly as described in the prompt.

```
Prepare a visual asset checklist for Devpost. Capture genuine screenshots from the running application: dashboard, AI assistant invoice extraction, invoice approval preview, PayPal Sandbox invoice result, invoice list/grid, reminder draft, analytics, and mobile layout. Do not generate fake screenshots or show placeholder data as a real integration. Use the PAYFLOW AI logo and editorial quiet-luxury palette. Devpost thumbnail must be JPG, PNG, or GIF, maximum 5 MB, recommended 3:2 ratio. Optimize image size and ensure legible wordmark at thumbnail scale. Store assets in docs/submission-assets/ and document which screen each image represents.
```

Do not proceed until these checks pass:

- Screenshots come from the actual running app.
- Thumbnail is 3:2 and under 5 MB.
- No misleading fake integration evidence.

PROMPT 37 — Three-minute YouTube demo script

Objective: Implement this phase exactly as described in the prompt.

```
Write and rehearse a demo video script under 3 minutes. It must show the product functioning on the device it was built for. Structure: 0:00–0:15 problem and product; 0:15–0:35 dashboard and AI assistant; 0:35–1:05 natural-language invoice extraction; 1:05–1:35 user review and approval; 1:35–2:00 real PayPal Sandbox invoice creation/sending; 2:00–2:25 verified status/webhook or clearly demonstrated sandbox state; 2:25–2:45 overdue reminder draft and approval; 2:45–2:58 impact and closing. Keep narration concise. Show real API activity only where safe; hide secrets and personal data. No copyrighted music or third-party trademarks without permission. Upload public or unlisted YouTube video and test the URL.
```

Do not proceed until these checks pass:

- Video is under 3 minutes.
- Shows actual working app.
- PayPal and AI use are visible.
- No secrets or unlicensed media appear.

PROMPT 38 — Devpost submission fact-check and finalization

Objective: Implement this phase exactly as described in the prompt.

```
Review the entire Devpost submission against the official hackathon rules and the actual implementation. Do not invent claims. Verify project name PAYFLOW AI, elevator pitch, project story, technologies, sponsor tools, repository URL, hosted demo URL, testing instructions, video URL, team name XyphX, and team membership. The team has two members; confirm the second member has accepted the Devpost invitation before final submission. For Additional Info: submitter type must match actual team setup; country of residence must be India if accurate; organization name should be XyphX only if representing it, otherwise N/A; project-new/existing answer must be truthful; PayPal/AI explanation must name the exact API and exact model/provider actually used; sponsor tools must include only those actually used; AI tools must be honestly listed. Confirm age/eligibility declarations for every team member. Do not guess PayPal integration experience or APIMatic/AG Grid feedback. Check repository is public and licensed. Confirm demo works in a fresh browser session. Produce a final submission checklist and list unresolved blockers. Do not click final submit without explicit human confirmation.
```

Do not proceed until these checks pass:

- Every claim matches the working implementation.
- Team and eligibility details are verified.
- Public repository, hosted demo, and video work.

- No final submission is made without user confirmation.

Optional improvement prompts — use only after the MVP works

PROMPT 39 — Natural-language correction and confirmation quality

Objective: Implement this phase exactly as described in the prompt.

```
Review the assistant's conversation UX. Add concise clarification questions when a required field is missing. Provide a structured summary before any write action. Make user edits easy. Prevent the model from silently changing amount, currency, customer, due date, or tax. Add explicit confirmation text such as 'This will create and send invoice INV-... through PayPal Sandbox after approval.' Keep AI messages concise and factual. Add tests for ambiguous and adversarial user instructions.
```

Do not proceed until these checks pass:

- No silent material changes.
- User sees exact action and values before approval.

PROMPT 40 — Operational observability and graceful failure

Objective: Implement this phase exactly as described in the prompt.

```
Add structured logs with request IDs/correlation IDs, safe error categories, and basic operational metrics for API errors, PayPal failures, AI provider failures, webhook verification failures, and scheduler failures. Do not log secrets or unnecessary PII. Add a small admin/developer diagnostics page only if it is protected and does not reveal credentials. Ensure the app remains usable when AI provider is unavailable: manual invoice creation should still work. Ensure dashboard remains usable if PayPal is temporarily unavailable, while clearly showing integration status. Document recovery steps.
```

Do not proceed until these checks pass:

- External service failures are visible and recoverable.
- Manual workflows still work without AI.
- No secrets/PII leak through observability.

PROMPT 41 — Final regression and release candidate

Objective: Implement this phase exactly as described in the prompt.

```
Freeze feature development. Run formatting, linting, type checks, unit tests, integration tests, frontend production build, backend startup, and full sandbox smoke test. Review all changed files and remove unused packages, dead buttons, debug prints, test credentials, placeholder text, and console errors. Verify mobile responsiveness, CORS, HTTPS, health checks, and Render environment variables. Tag a release candidate in Git. Produce a concise release report: completed features, test results, known limitations, sponsor integrations actually used, deployment URLs, and any submission blockers. Do not add new scope unless it fixes a blocker.
```

Do not proceed until these checks pass:

- Production build succeeds.
- Critical workflow passes from clean session.
- Known limitations are documented.
- Release candidate is reproducible.

Execution order and time-boxing

Recommended order

Run prompts 1–24 first. This delivers the essential product. Then run prompts 25–30 only for sponsor integrations and deployment that fit the remaining time. Finish with prompts 31–38. Prompts 39–41 are optional polish/release passes.

Time window	Focus	Must be true by end
First 6–8 hours	Scaffold, data, auth, customers, invoices	App runs; invoice draft can be created
Next 5–7 hours	PayPal Sandbox OAuth, invoice create/send	Approved invoice reaches PayPal Sandbox
Next 4–6 hours	AI tool calling and invoice extraction	Natural-language request creates draft
Next 4–6 hours	Approval, webhooks, reminder drafts	Status sync and safe reminder flow
Final 6–8 hours	UI polish, tests, deployment, video, Devpost	Judge-ready end-to-end demo

Important: if time becomes tight, cut optional sponsor integrations before cutting PayPal invoice creation, AI tool calling, human approval, webhook verification, or the demo. A coherent product with two real integrations is stronger than six superficial integrations.

Definition of done

- A fresh user can register and sign in.
- A user can add a customer and create an invoice draft.
- The AI can parse a natural-language invoice instruction into a validated, editable draft.
- The user explicitly approves the invoice.
- The backend creates and sends the invoice using PayPal Sandbox.
- PayPal response identifiers and URLs are stored accurately.
- Verified PayPal webhooks or status retrieval synchronize invoice state idempotently.
- AI can find overdue invoices and prepare reminder drafts based on actual records.
- The dashboard displays accurate, currency-aware totals.
- The UI follows the editorial quiet-luxury specification.
- The public repository contains source, README, .env.example, tests, and an OSI-approved license.
- The hosted demo and under-three-minute YouTube video work.
- Devpost fields and sponsor claims are truthful and complete.

Final operating instruction for Antigravity

```
Work incrementally. Before each prompt, inspect the current code and preserve working behavior. After each change, run the relevant tests and report: files changed, decisions made, commands executed, test results, manual checks still needed, and any blockers. Do not pretend a command succeeded if it was not run. Do not silently skip acceptance criteria. If an API detail is uncertain, consult official documentation or ask for clarification rather than guessing. Prefer the smallest implementation that is secure, understandable, and demonstrably working.
```

PAYFLOW AI • TEAM XYPHX • END OF PLAYBOOK